Making them refuse

Last week you wrote classes that accept anything. By the end of this session your three classes cannot be put into an invalid state, even after deliberately trying to break them.

Warm-up. The invariant list

For each of your classes, write every field down one column. Beside each field write **the one thing that must always be true about it**, from the moment the object exists until it is thrown away.

```
Student
  studentId       not blank, never changes after creation
  name            not blank
  gpa             between 0.0 and 4.0
  enrolmentYear   between 2000 and this year, never changes
```

Then a second pass. Beside each line, write **where the rule has to be enforced**. Some are the constructor only. Some are the constructor and a method. Some need no enforcement because the field can never change and the type already limits it.

Harden them

Three passes over each class, in this order. Do not mix them, because doing all three at once on one class is how you end up with a file that does not compile and no idea which change broke it.

Pass 1. Lock what never changes

Add `final` to every field your list says never changes. `studentId`, `enrolmentYear`, `employeeId`, `courseCode`.

Then compile. If the compiler complains, you have a setter you should not have or an assignment outside the constructor. Both are the compiler finding a design problem for you.

Delete any setter that now fails to compile. Do not work around it.

Pass 2. Make the constructor a gate

Add the checks from your list. Bad data in, no object out.

```
if (gpa < 0.0 || gpa > 4.0) {
    throw new IllegalArgumentException("GPA must be between 0 and 4: " + gpa);
}
```

Put the bad value in every message. `"Invalid GPA"` tells you nothing at 2am. `"GPA must be between 0 and 4: 9.5"` tells you where to look.

You will write the same "not blank" check in six places. That is a signal. Write it once as a private helper and call it from everywhere, as the lecture did with `requireText`. It takes no fields, so it should be `static`.

Then the methods that change things. `updateGpa` must enforce the same rule as the constructor, because an object that is born valid and can be made invalid afterwards was never protected at all.

Pass 3. Audit the public surface

This is the pass people skip and it is the one that teaches design.

List every `public` method on each class. For each one, ask: **is this something the thing genuinely does?**

A student updates a GPA. A student does not have a GPA assigned by a stranger. A professor completes another year. A professor does not have their experience overwritten.

Delete the ones that fail. Deleting code you wrote last week is uncomfortable and it is the actual work. Make a note of each deletion for the checklist.

Then: let them issue their own ids

Add a `static` counter to `Student` and `Professor`, as `Account` did in the lecture. The object produces its own identifier and the constructor no longer takes one.

```
private static int lastIdIssued = 1000;
```

Your constructor signature changes, which means `UniversityApp` changes too. That is expected. The specification describes what the object knows, not what the constructor asks for, and this is an improvement: nobody outside can pick an id, so nobody outside can pick one that is already taken.

> **Mistake: remember what this counter cannot do.** It lives in memory. Close the program and it resets to 1000.
>
> In week eleven you will save students to a file and load them back, and the next student you create will be handed an id that already belongs to somebody. That is not a bug in what you wrote today. It is the honest limit of a counter, and you should be able to say so out loud.

Break them

The best half of the session.

Write a new class called `BreakIt`, with a `main`, whose only job is to put your objects into invalid states. Try everything you can think of.

```
new Student("", 3.9, 2024);
new Student("Ada", -1.0, 2024);
new Student("Ada", 3.9, 1850);
new Student(null, 3.9, 2024);
new Course("CS101", "Programming", 0, "Fall 2026");
```

Then the interesting ones, which are not about the constructor:

- Create a valid student, then call every method that changes something with a bad value.
- Create a valid professor, then call `completeAnotherYear()` two hundred times. Is there an upper limit? Should there be?
- Set a name to a string of forty spaces. Does `isBlank()` catch that? Check.
- Give a course a title of `""` after construction, if you left a setter that allows it.

Score yourself. One point for every line that succeeds in creating or producing an invalid object. A perfect score is zero.

Then fix what you found, and run `BreakIt` again until it scores zero.

> **Note: working next to somebody.** If there is somebody beside you, swap. Hand them your three classes and take theirs, then write `BreakIt` against code you did not write.
>
> You will find more in theirs than you found in your own, and they will find more in yours. That is not a comment on either of you. It is the reason professional code is reviewed by somebody else.

Keep `BreakIt` in the repository. It is the closest thing you have to a test suite until week ten, and in week ten you will convert it into one.

Commit

```
git add .
git commit -m "Week 3: invariants enforced, ids generated, BreakIt scores zero"
git push
```

Before you leave

- [ ] Every field that never changes is `final`
- [ ] Every constructor refuses bad data with a message containing the bad value

- [ ] Every method that changes a field enforces the same rule as the constructor
- [ ] The "not blank" check exists once, as a `static` helper
- [ ] `Student` and `Professor` issue their own ids
- [ ] You deleted at least one method you wrote last week
- [ ] `BreakIt` exists, is committed, and scores zero

Going further

An immutable class. Write `Semester` with a term and a year, both `final`, no setters, and a `toString` that returns `"Fall 2026"`. Then replace the `String semester` field on `Course` with it. Notice that you can now hand a `Semester` to anything without wondering what it will do to it.

The rule you cannot enforce yet. "A student cannot register for more than six courses." Where does it belong, and what does that class need to know? It cannot know it yet. Write down what is missing. You add it next week.

A validation message somebody else can read. Your exception messages are written for you. Rewrite one of them for a user of a system who does not know your field names. Decide whether that is really the right place for such a message, and if not, whose job it is.